

Reviewer Pack — Bounded Arithmetic Proof Length and Circuit Complexity for T...

Entry 94191613e108 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 03:27:05 UTC

Bounded Arithmetic Proof Length and Circuit Complexity for Tautologies

Entry ID: 94191613e108 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 03:27:05 UTC

1. Conjecture

Field A (mathematical branch): Bounded Arithmetic (Cook-Reckhow-Krajicek) **Field B** (complexity object): Proof Length in Extended Frege System

Statement:

For any tautology in CNF with n variables, the minimal proof length in the Extended Frege system is $\Theta(\text{circ_size})$, where circ_size is the minimal circuit size required to compute the tautology.

Rationale (proposer's reasoning):

Bounded arithmetic theories like S_{12} can simulate proof systems, and their proof complexity might be tied to the circuit complexity of the formulas they prove. By linking proof length to circuit size, we might uncover structural properties that separate bounded arithmetic theories.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d38e3e145a89d327

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = [[random.choice([-i, i]) for _ in range(random.randint(1, n))] for _ in range(n)]
        return cnf

    def is_tautology(cnf):
        # Simplified DPLL algorithm to check if a CNF formula is tautological
        def dpll(clauses, assignment):
            if not clauses:
                return True
            unit_clause = next((c for c in clauses if len(c) == 1), None)
            if unit_clause:
                literal = unit_clause[0]
                new_assignment = assignment.copy()
                new_assignment[literal] = True
                if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment):
                    return True
                new_assignment[literal] = False
                if dpll([c for c in clauses if -literal not in c], new_assignment):
                    return True
                return False
            pure_literal = next((l for l in range(1, n+1) if all(l not in c and -l not in c for c in clauses)), None)
            if pure_literal is not None:
                new_assignment[pure_literal] = True
                if dpll([c for c in clauses if pure_literal not in c], new_assignment):
                    return True
                new_assignment[pure_literal] = False
                if dpll([c for c in clauses if -pure_literal not in c], new_assignment):
                    return True
                return False
            literal, _ = random.choice(clauses)
            new_assignment[literal] = True
            if dpll([c for c in clauses if literal not in c], new_assignment):
                return True
            new_assignment[literal] = False
```

```

        if dpll([c for c in clauses if -literal not in c], new_assignment):
            return True
        return False
    return dpll(cnf, {})

def circuit_size(cnf):
    # Simplified SAT-based minimizer to estimate circuit size
    # This is a placeholder and should be replaced with an actual algorithm
    return len(cnf) * len(cnf[0])

def extended_frege_proof_length(cnf):
    # Simplified DPLL-style Extended Frege proof length estimator
    # This is a placeholder and should be replaced with an actual algorithm
    return len(cnf) ** 2

n = random.randint(5, 40)
cnf = generate_cnf(n)

if not is_tautology(cnf):
    return {
        "metric_name": "proof_length_to_circ_size_ratio",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "not a tautology"
    }

circ_size = circuit_size(cnf)
proof_length = extended_frege_proof_length(cnf)

return {
    "metric_name": "proof_length_to_circ_size_ratio",
    "metric_value": proof_length / circ_size,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra

```

```

else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample='not a tautology' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_547f5a7e.py", line 99, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_547f5a7e.py", line 71, in run_trial
    cnf = generate_cnf(n)
          ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_547f5a7e.py", line 22, in generate_cnf
    cnf = [[random.choice([-i, i]) for _ in range(random.randint(1, n))] for _ in range(n)]
              ^
NameError: name 'i' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to NameError before collecting data, preventing evaluation of support fraction or counterexamples. | next: Fix the NameError in generate_cnf by defining 'i' (likely a typo for 'n') and rerun tests with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	79530
2	preregistration	ollama_remote	qwen3:8b	0	0	24266
3	novelty	ollama_remote	qwen3:8b	0	0	20964
4	novelty	ollama_remote	qwen3:8b	0	0	15820
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14218
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11521
7	judge	ollama_remote	qwen3:8b	0	0	16439

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 182759 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/94191613e108.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/94191613e108.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/94191613e108.tar.gz (if generated)